

ECE 379K Design Document

1 Document Metadata

Provide project title, team members, course info, date, and version. Add a revision history table and role descriptions (Technical Lead, Project Manager, Concurrency Specialist, etc.). Include schedules of what role each team member will play throughout the semester (if you are all going to share each role, state that here).

1.1 Version History

Version	Date	Owner	Description
v1.0	10/20/2025	Andres	Rough Draft
v1.1	10/22/2025	Maanav and Kareem	Final Draft

1.2 Team Members

- Maanav Koladia
- Karemeldin Mohamed
- Shrikar Thangavel
- Andres Wearden

1.3 Team Roles and Responsibilities

All team members will contribute collectively to every stage of the project, including design, implementation, testing, and documentation. Rather than assigning fixed roles, responsibilities will be distributed dynamically based on project requirements and individual expertise. This collaborative approach ensures that all members gain experience across the full development cycle and that the project benefits from diverse perspectives and skill sets.

2 Executive Summary

This project’s goal is to create a high-performance, real-time 3D fluid simulator that is both visually realistic and numerically stable. The core problem being solved is simulating an incompressible fluid (like water) in a way that conserves mass—preventing it from ”squishing” or ”expanding”—while running at interactive frame rates. The overall approach is a decoupled, three-service architecture built in C. First, a physics service (FS-3D) uses CUDA on the GPU to perform the heavy computational work of solving the fluid dynamics equations on a 3D grid. Second, a Transform service, parallelized across a CPU thread pool, consumes this physics data and converts the fluid’s 3D state (an implicit levelSet grid) into an explicit triangle mesh via the Marching Cubes algorithm.

This generated mesh is then passed to a third, independent GL/Display service, which uses OpenGL to render the fluid with realistic water shaders. By decoupling these three stages with multi-threaded MPMC (Multi-Producer, Multi-Consumer) queues, the system avoids bottlenecks and allows the physics simulation to run at a fixed, stable 60 steps per second independent of the final render frame rate. The intended outcome is a stable, interactive live demo of a 3D water simulation that achieves a between 20-30 FPS, with the architecture designed to be directly scalable from its initial 2D milestone.

3 System Overview

3.1 Overview

This project’s goal is to create a high-performance, real-time 3D fluid simulator that is both visually realistic and numerically stable. The core problem being solved is simulating an incompressible fluid (like water) in a way that conserves mass—preventing it from ”squishing” or ”expanding”—while running at interactive frame rates.

3.2 System Architecture

The overall approach follows a decoupled, three-service architecture implemented in C. The major components are as follows:

1. **Physics Service (FS-3D)** — Uses CUDA on the GPU to perform the heavy computational work of solving the fluid dynamics equations on a 3D grid.
2. **Transform Service** — Parallelized across a CPU thread pool. It consumes the physics data and converts the fluid’s 3D state (an implicit level set grid) into an explicit triangle mesh using the Marching Cubes algorithm.
3. **GL/Display Service** — Uses OpenGL to render the generated mesh with realistic water shaders.

3.3 Execution Model

The three stages communicate via multi-threaded MPMC (Multi-Producer, Multi-Consumer) queues. This decoupled design eliminates synchronization bottlenecks and allows the physics simulation to maintain a fixed, stable rate of 60 simulation steps per second, independent of the rendering frame rate.

3.4 Expected Outcome

The intended outcome is a stable, interactive live demo of a 3D water simulation achieving between 20–30 FPS. The architecture is designed to be scalable and directly extendable from the project’s initial 2D milestone implementation.

4 Functional Requirements

Provide a numbered list of functional requirements. Each should be specific, measurable, and testable.

5 Non-Functional Requirements

Detail performance, scalability, reliability, maintainability, and portability expectations. Provide measurable targets where possible.

6 Detailed Design

Expand on the module decomposition. For each module, describe its responsibilities, inputs/outputs, and interactions. Include diagrams, pseudocode, and data structures where appropriate.

7 Performance & Concurrency Plan

Explain how work is partitioned across CPU and GPU. Describe concurrency strategies, bottleneck mitigation, load balancing, and plans for benchmarking.

8 Testing & Validation

Describe your strategy for unit, integration, performance, and regression testing. Identify specific test cases and how success will be measured.

9 Risks & Mitigations

Provide a risk table listing risks, their likelihood, impact, and mitigation strategies. Include contingency plans for the most critical risks.

10 Project Management

Detail the project timeline, milestones with dates, and responsibilities. Describe workflow practices such as Git branching strategy, code reviews, and CI/CD pipeline.

11 References

List all relevant references, including research papers, documentation, and libraries used. Follow a consistent citation format.

12 Appendix

Include supporting materials such as diagrams, pseudocode, glossary of terms, and any other resources that support your design.